

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Industry Problem Solving Task

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	

Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	

Annexure A

Industry Problem Solving Task

1: Automated Defect Detection in Manufacturing

A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.

Questions:

1. How can PCA-based shape priors and contour-based representations be used to model standard component shapes?
2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.
3. Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.

2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labeled data for these obstacles is limited.

Questions:

1. Explain how data augmentation using GANs can help generate synthetic images for training.
2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.
3. Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.

Automated Defect Detection in Manufacturing

1. PCA-based shape priors and contour-based representations

The first step is to create a **standard shape model** of a defect-free mechanical component.

PCA-based Shape Prior:

1. Collect images of many **defect-free components**.
2. Segment each component from the background.
3. Extract its contour/boundary points.
4. Align all shapes using translation, rotation, and scaling.
5. Represent each contour as a vector of coordinates.
6. Apply **Principal Component Analysis (PCA)** to these shape vectors.
7. PCA identifies the **mean shape** and the main modes of normal variation caused by manufacturing tolerances.
8. A new component can be reconstructed using the PCA model. A large difference between the actual contour and reconstructed normal shape indicates a possible defect.

For example:

Normal shapes → **Alignment** → **Contour extraction** → **PCA** → **Mean shape + variation modes** → **Shape prior**

Contour-based representation can use:

- Boundary coordinates
- Chain codes
- Curvature
- Fourier descriptors
- Shape signatures

These representations capture the geometry of the component while being less dependent on its internal texture.

2. Machine-learning/deep-learning pipeline

A suitable automated pipeline is:

**Image Acquisition → Preprocessing → Segmentation → Feature Extraction
→ Classification → Defect Decision**

Step 1: Image acquisition

Capture images using a fixed industrial camera and controlled setup.

Step 2: Preprocessing

- Convert to grayscale
- Remove noise using Gaussian/median filtering
- Normalize intensity
- Correct illumination

Step 3: Segmentation

Separate the mechanical component from the background using:

- Thresholding
- Edge detection
- Background subtraction

Step 4: Feature extraction

Two approaches can be used:

Traditional ML:

- Extract contour/shape features
- PCA features
- Texture features
- Edge features
- Area, perimeter, circularity, etc.

Then train an **SVM**:

Features → SVM → Defective / Non-defective

A **GMM (Gaussian Mixture Model)** can model the distribution of normal features.

If a new component has a very low probability under the normal distribution, it can be classified as defective.

Deep learning:

A CNN can automatically learn useful features directly from images:

Input Image → CNN → Feature Extraction → Fully Connected Layer → Defective / Non-defective

A CNN is generally preferable when a sufficiently large labeled dataset is available.

3. Handling noise, occlusion, and illumination

A. Noise

Industrial images may contain sensor noise, dust, or small unwanted structures.

Methods:

- **Gaussian filter** – reduces Gaussian noise.
- **Median filter** – effective for salt-and-pepper noise.
- Morphological opening/closing – removes small unwanted regions and fills gaps.
- Canny edge detection can be used after smoothing.

B. Partial occlusions

Parts may be partially hidden by fixtures, other components, or debris.

Methods:

- Use **local contour features** instead of relying only on the complete contour.
- Use feature descriptors that remain useful when part of the object is missing.
- Train CNNs using artificially occluded images.
- Use PCA shape priors to estimate/reconstruct the missing portion.
- Use multiple camera views when possible.

C. Varying illumination

Different lighting can change pixel intensity even when the component is defect-free.

Methods:

- Use controlled and uniform industrial lighting.
- Apply intensity normalization.
- Use histogram equalization or CLAHE.

- Use adaptive thresholding instead of a fixed threshold.
- Use illumination-invariant features such as edges, contours, and normalized gradients.
- During CNN training, use **data augmentation** with different brightness and contrast levels.

Overall approach

A strong practical solution would be:

Camera → Noise removal → Illumination correction → Segmentation → Contour extraction → PCA shape features → SVM/GMM classifier → Defect detection

For a large labeled dataset, the alternative is:

Camera → Preprocessing → CNN → Defective / Non-defective

This system can automatically identify defects such as **dents, cracks, missing portions, dimensional deviations, and abnormal shapes** while tolerating small variations caused by normal manufacturing processes.

2. Autonomous Vehicle Obstacle Recognition 🚗

1. Data augmentation using GANs

Since **rare obstacles have limited labeled data**, GANs (Generative Adversarial Networks) can generate realistic synthetic training images.

A GAN consists of two networks:

- **Generator:** Creates fake obstacle images.
- **Discriminator:** Tries to distinguish real images from generated images.

They compete with each other until the generator produces realistic images.

Process:

Real obstacle images → GAN training → Synthetic obstacle images → Add to

training dataset → Train detector

For example, if there are very few images of an unusual object such as a fallen object, animal, or damaged vehicle, a GAN can generate variations with different:

- Positions
- Sizes
- Lighting conditions
- Backgrounds
- Viewing angles
- Occlusions

This increases dataset diversity and helps reduce **overfitting**.

However, synthetic images should be validated carefully because unrealistic GAN-generated objects can introduce incorrect patterns into the training data.

2. CNN vs Vision Transformer

Feature	CNN	Vision Transformer (ViT)
Basic operation	Convolution	Self-attention
Local features	Excellent	Good
Small objects	Very good with multi-scale features	Can be good, but depends on resolution/patch size
Global context	Limited/requires deeper layers	Excellent
Training data	Can work well with smaller datasets	Usually benefits from large datasets/pretraining
Computation	Generally lower	Generally higher
Embedded deployment	Easier	More demanding

CNNs:

CNNs are very effective for small and sparse objects because convolution

captures **local edges, textures, and shapes**. Models such as YOLO-style detectors and feature pyramids can detect objects at multiple scales.

Vision Transformers:

Transformers use self-attention to understand relationships between distant regions of an image. This can help when an obstacle is small but its context is important. However, high-resolution images and attention operations can require significant computational resources.

Conclusion:

For an autonomous vehicle with **limited labeled data and real-time requirements, CNN-based detectors are generally the safer choice**, especially when using a lightweight architecture. A pretrained/hybrid Transformer can be considered when sufficient computing power and training data are available.

3. Real-time inference pipeline

A practical embedded-system pipeline is:

Camera/LiDAR → Preprocessing → Object Detection → Post-processing → Tracking → Decision

Step 1: Sensor input

Capture frames from cameras, and optionally combine them with LiDAR/radar.

Step 2: Preprocessing

- Resize image
- Normalize pixel values
- Remove unnecessary regions
- Convert to the model's required input format

Step 3: Lightweight detection model

Use a lightweight CNN detector such as a **YOLO-style model** or another optimized object detector.

The model produces:

Bounding Box + Object Class + Confidence Score

Step 4: Post-processing

Use **Non-Maximum Suppression (NMS)** to remove duplicate detections.

Step 5: Object tracking

Track detected obstacles across consecutive frames using a lightweight tracker.

Step 6: Decision

Estimate:

- Obstacle position
- Distance
- Movement
- Collision risk

The vehicle can then **brake, slow down, or change direction**.

Computational optimization

For embedded systems:

- Use a lightweight CNN.
- Reduce input resolution when possible.
- Apply **quantization** such as INT8.
- Use pruning/model compression.
- Use GPU/NPU acceleration.
- Process only relevant regions when appropriate.
- Maintain a suitable FPS rather than maximizing image resolution.

Final architecture

Camera → Resize/Normalize → Lightweight CNN Detector → NMS → Object Tracking → Distance/Risk Estimation → Vehicle Action

This approach provides a good balance between **accuracy, detection of rare obstacles, and real-time performance** in an embedded autonomous vehicle.

